

MINISTRY OF EDUCATION AND RESEARCH

OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova

Faculty of Computers, Informatics and Microelectronics

Department of Software Engineering and Automatics

REPORT

Laboratory work No. 1
Network Programming

Concurrency in Practice

Authors: **Pancenco Mihail**

Elciscev Andrey

Group: **FAF-243**

Supervisors:

asist. univ. Fiștic Cristofor

asist. univ. Bîtca Ernest

DISA, FCIM, UTM

Chișinău – 2026

Table of Contents

1 Introduction and experimental platform **1**
1.1 Purpose of the laboratory work 1
1.2 Hardware and software 1
2 Software architecture and UML models **2**
2.1 Responsibilities and process boundaries 2
2.2 UML class diagram: coordination and metrics 3
2.3 UML sequence diagram: measure first, visualise later 5
2.4 How the architecture affects the experiment 6
3 Input data, correctness, and measurement method **6**
3.1 Generating and checking the dataset 6
3.2 What exactly we measure 7
3.3 Speed-up and interpretation 8
4 Task 1a: partitioning one file among threads **9**
4.1 Implementation and boundary ownership 9
4.2 Source code: worker creation, boundaries, and counting 9
4.3 Measured results 12
4.4 Answers concerning scaling and memory 13
5 Task 1b: shared counters, locking, and local counters **14**
5.1 Three implementations of the same counting operation 14
5.2 Source code: all three counting methods 14
5.3 Measurements and correctness 17
5.4 How an update is lost 18
6 Task 1c: a producer and consumers **19**
6.1 Pipeline, batching, and termination 19
6.2 Source code: queue lifecycle and both pipeline stages 19
6.3 Measured results and comparison 22
7 Task 2: eight files on the USB flash drive **24**
7.1 Implementation and cold-cache procedure 24
7.2 Source code: file assignment, buffering, and parsing 24
7.3 Measured results 27
7.4 Answers about storage and concurrency 27
8 Answers to all control questions **29**
9 Build, run, and inspect the implementation **31**
9.1 Source organisation and command-line entry point 31
9.2 Measuring a processing command on macOS 32
9.3 USB preparation and access 32
9.4 Dashboard and evidence 32
10 Conclusions **33**
Bibliography **34**

1 Introduction and experimental platform

1.1 Purpose of the laboratory work

In this laboratory work, we investigate how the number of operating-system threads and the way we distribute work affect execution time, correctness, and memory consumption. We process the same 50 million integers throughout the experiments. This gives us a common workload for comparing direct file partitioning, shared counters, per-thread counters, a producer–consumer queue, and reading complete files from removable storage.

Our aim is to explain the measurements rather than assume that more threads must make a program faster. We distinguish concurrency, which lets several activities make progress, from parallel execution, which requires available processing resources. We also distinguish a short execution time from a correct result: an implementation that loses most of its updates is not a successful optimisation.

The laboratory handout defines Tasks 1a, 1b, 1c, and 2, together with eight task questions and six control questions [1]. We answer all of them explicitly below. We include actual measurements, normalised speed-up, peak resident memory, selected source excerpts, and instructions for reproduction. The artificial values used during earlier dashboard testing are excluded from every experimental chart and table in this report.

1.2 Hardware and software

Table 1.1 - Experimental environment, recorded on 25 October 2026.

Component	Recorded configuration
Computer	MacBook Air, model identifier Mac15,13; Apple M3
Physical CPU cores	8: four performance cores and four efficiency cores
Logical processors	8, confirmed by macOS and `Environment.ProcessorCount`
Memory	16 GB unified memory
Internal storage	APPLE SSD AP0512Z, approximately 500.28 GB, NVMe; APFS
USB flash drive	Wilk “USB DISK 3.0”, controller vendor Phison; volume `ESD-USB`, approximately 7.93 GB
USB connection	SuperSpeed USB 3.0 device through a GenesysLogic USB 3.1 hub; the negotiated SuperSpeed connection is not a measured transfer rate
USB filesystem	MS-DOS FAT32; mounted at `/Volumes/ESD-USB`
Operating system	macOS 14.7, build 23H124, ARM64
Language/runtime	C#, .NET SDK 10.0.400; .NET runtime 10.0.11
Threading	Explicit `System.Threading.Thread` workers; bounded `BlockingCollection` in Task 1c

The M3 contains two types of physical core. An efficiency core is still a real core, but we should not expect it to process this workload at the same rate as a performance core. The machine exposes one logical processor per physical core, so eight logical processors do not mean sixteen hardware execution contexts. Apple's technical specification confirms the four-plus-four CPU configuration [2].

We recorded the USB model, capacity, filesystem, and negotiated connection using macOS device information. We did not infer the flash drive's performance from the words "USB 3.0": the flash controller, NAND memory, hub, and access pattern can all limit the measured application throughput.

2 Software architecture and UML models

2.1 Responsibilities and process boundaries

We separate user interaction, experiment coordination, counting algorithms, measurement storage, and visualisation. The separation is implemented by concrete classes and separate file-based programs; it is not an inheritance hierarchy or a framework of interchangeable interfaces. Table 2.1 maps those responsibilities to the submitted source files.

Table 2.1 - Architecture of the implemented laboratory application.

Element	Location / lifetime	Responsibility
`Program`	`Program.cs`; controller process	Creates the collectors and application, reads menu choices, and launches the Drawer for option `d`.
`Application`	`Program.cs`; controller process	Generates input; runs each use case and configuration; caches Task 1b's array; coordinates measurement and error checks.
Four metric collectors	`Program.cs`; controller process	Retain per-configuration metric records in memory and export the lists for plotting.
`TaskOneB`	`TaskOneB.cs`; called directly	Implements the three counting functions over the preloaded, read-only integer array; returns `SignCounts`.
Task 1a / 1c / 2 programs	Separate file-based executables	Create their own worker threads, read the selected input, print sign totals, and return an exit status.
`Drawer`	`Drawer.cs`; separate process	Receives a JSON snapshot, creates HTML/SVG charts and tables, and opens the saved report in a browser.
Reproduction entry point	`documentation/reproduce/`	Selects tasks by command-line parameters and provides the supplementary Task 1b benchmark used in this report.

The controller is the program whose menu we operate. Tasks 1a, 1c, and 2 run as child processes for each measured configuration. Each child can itself create multiple OS threads; a process boundary and a thread boundary are therefore different parts of our design. The three task files each declare a class named

`Program`, which is valid because they are built separately, not combined into one assembly with competing entry points.

Task 1b is different: the controller calls the static `TaskOneB` methods directly so that it can load the array before starting the counting stopwatch. The supplementary benchmark preserves the same distinction between loading and counting, but runs independently to provide additional one-thread references and whole-process memory measurements.

2.2 UML class diagram: coordination and metrics

Figure 2.1 shows the main controller classes and the four concrete collector associations. A solid navigable association means that `Application` keeps a reference to one collector of that type. The dashed dependency marked *create* means that `Program.Main` constructs `Application`; the same method constructs and supplies the four collectors through the application constructor. We use plain associations, rather than composition diamonds, because the application receives those objects instead of constructing or exclusively owning them.

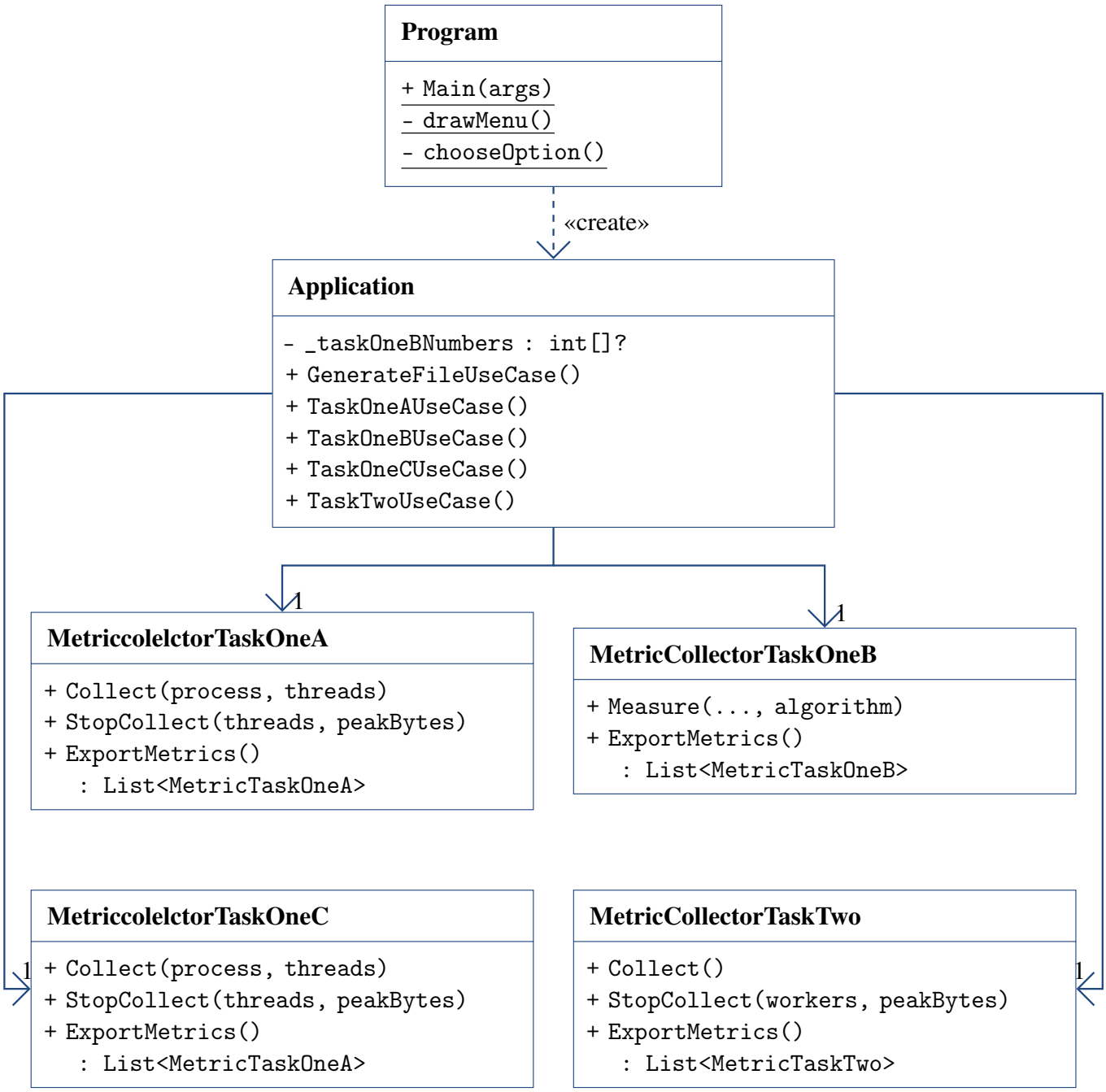


Figure 2.1 - UML class diagram of the controller and metric collectors. Selected members are shown; parameter lists are shortened where marked by dots. A plus sign denotes public access and a minus sign denotes private access. Underlined operations are static.

The diagram deliberately preserves the current source names, including `MetriccolelctorTaskOneA` and `MetriccolelctorTaskOneC`. The Task 1c collector currently exports `MetricTaskOneA` records: it reuses their thread/time/memory data shape. Although a separate `MetricTaskOneC` record is declared in the source, this collector does not use that type. We describe the implementation as it exists rather than introduce a class relationship that is absent from the code.

The records represent measurements, not running tasks. Task 1b records retain the algorithm version, repetition, worker count, time, and all three sign counters. The other active metric records retain a worker/thread count, elapsed time, and peak-memory value; their task programs print sign totals separately. A successful child exit is checked by the menu, while the report's reproduction and validation procedure additionally checks the actual printed counters against the dataset reference.

2.3 UML sequence diagram: measure first, visualise later

Figure 2.2 follows one Task 1a configuration and the later drawing request. The other file-based tasks follow the same launch–wait–collect structure. The diagram's vertical dashed lines are participant lifelines; solid arrows show calls or actions, and dashed arrows show completion or returned information. The narrow rectangles mark periods of execution. The collector is an object in the controller process, while the task and Drawer lifelines represent separate child processes.

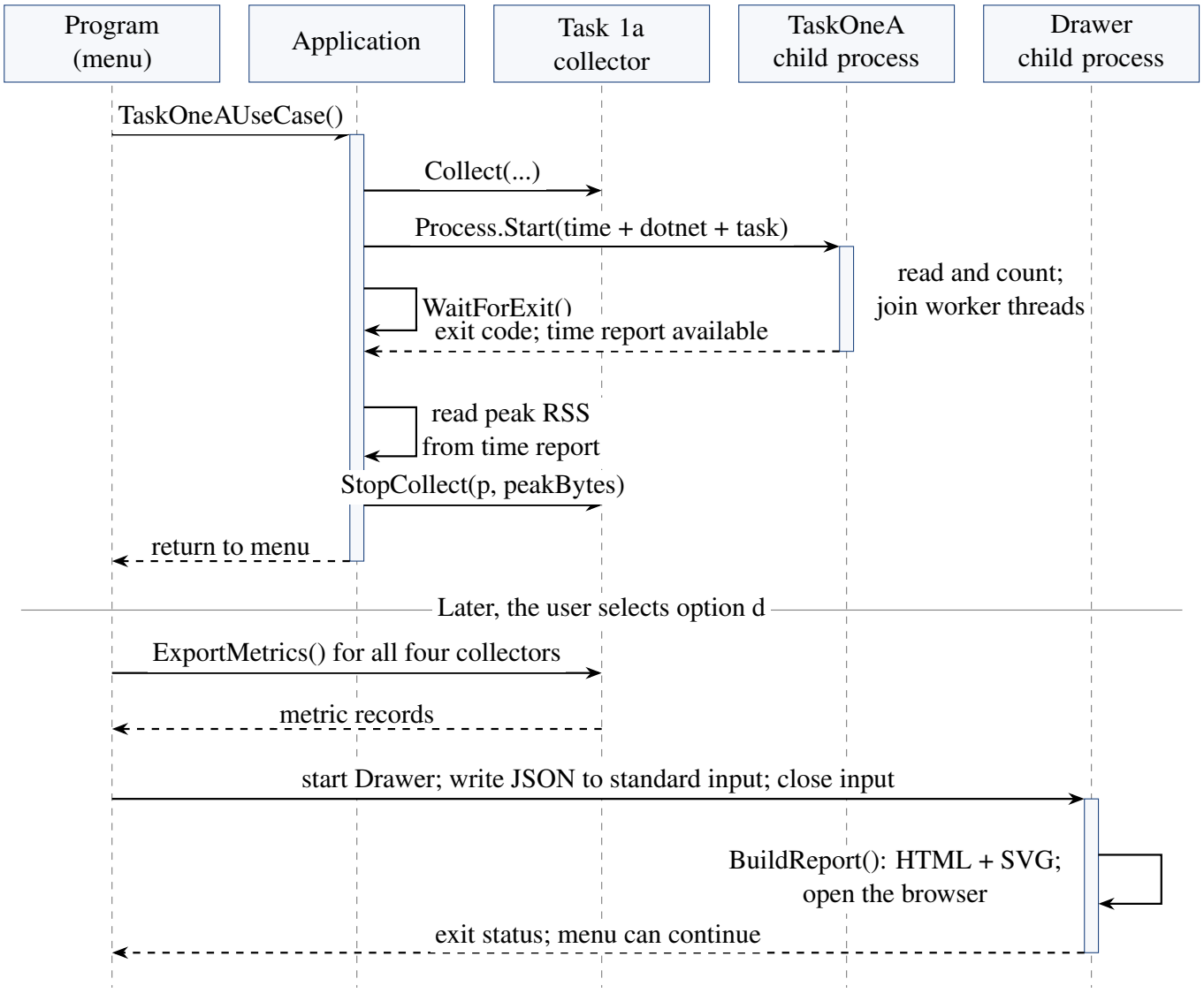


Figure 2.2 - UML sequence diagram for a measured Task 1a run followed by option `d`. The export step represents all four collectors; the browser itself is outside the displayed participants.

First, the collector starts the outer stopwatch. The application then launches `/usr/bin/time` around the appropriate `dotnet` task command, waits for completion, checks its exit code, and reads the peak RSS from the time-report file. Finally, it stops collection and saves a metric record. The peak-memory value used by these use cases comes from that report; the presence of a memory-tracking helper elsewhere in a collector does not mean that helper is the active measurement path.

When we later select `d`, `Program` exports the four metric lists, serialises their values as JSON, and writes the snapshot to the Drawer's redirected standard input. Closing that input is part of the protocol: `Drawer.Main` reads until the end of the stream before deserialising. It builds the HTML report, saves it, opens the browser, and returns an exit status. The controller can then display its menu again without waiting for the browser tab to close.

The snapshot is independent of the counting algorithms. Browser checkboxes select which Task 1b runs are drawn and recalculate chart scales; they do not rerun benchmarks or modify the original measured counters. Choosing `d` again creates a fresh snapshot from the controller's current in-memory records.

2.4 How the architecture affects the experiment

A fresh worker process gives each file-based configuration its own runtime state and peak-RSS high-water mark. It avoids attributing a previous configuration's peak resident memory to the next one. It does not reset the operating system's filesystem cache, which is why the USB reconnect procedure remains necessary.

The controller's cached Task 1b array and the browser live outside a Task 1a/1c/2 child process. Their memory is therefore not part of that child's peak RSS. Conversely, the supplementary Task 1b benchmark loads its array within its own process, so its external process peak includes loading even though its counting stopwatch excludes it. These process boundaries explain the memory scope used throughout the results.

Thread ownership differs inside each algorithm. Task 1a assigns byte ranges before starting workers; Task 1b shares a read-only array but varies ownership of the counters; Task 1c transfers batches through a bounded queue; Task 2 assigns whole-file intervals. We keep those choices inside their task implementations rather than letting the menu manage individual numbers, lines, or queue operations. The following task chapters show the actual source code implementing each choice.

3 Input data, correctness, and measurement method

3.1 Generating and checking the dataset

We generate `numbers.txt` with $N = 50\,000\,000$ random integers in the inclusive interval $[-1\,000\,000, 1\,000\,000]$, one integer per line. The call to `Random.Next(minValue, maxValue + 1)`

makes the upper limit inclusive. While generating the main file, we also write the same numbers, in the same order, into eight consecutive part files. Each part contains exactly 6 250 000 lines.

The input used in the measurements occupies 369 439 934 bytes, approximately 352.33 MiB. We checked every line count and computed SHA-256 digests. Concatenating `numbers1.txt` through `numbers8.txt` reproduces the main file byte for byte. The manifest in `data/dataset_manifest.json` records the exact sizes and hashes. The generator does not store its random seed, so the existing input and its hashes identify this experiment; generating a new input may produce different sign totals.

```
int number = random.Next(minValue, maxValue + 1);
mainWriter.WriteLine(number);
partWriter.WriteLine(number);
```

Listing 3.1.1 - Generating the same ordered data for the main file and the eight parts.

For the current dataset, the correct result is

$$N_- = 24\,996\,544, \qquad N_+ = 25\,003\,426, \qquad N_0 = 30, \tag{3.1.1}$$

and therefore

$$N_- + N_+ + N_0 = 50\,000\,000. \tag{3.1.2}$$

A separate validation pass confirmed the reference counters for all seven Task 1a and all seven Task 1c configurations; these validation-pass timings are archived but are not substituted into the earlier performance curves. We require agreement in all three counters, not only in their sum. Equality of a sum alone could hide compensating mistakes. We use the correct per-thread result and the locked implementation as independent implementation checks, and retain validation output from the file-based tasks. The deliberately unsynchronised eight-thread version is expected to fail this check.

3.2 What exactly we measure

For Tasks 1a and 1c, the saved application sessions measure elapsed wall-clock time around launching and waiting for each processing program. File opening, reading, parsing, thread creation, joining, and process/runtime startup are included. We exclude data generation, compilation, chart generation, and report preparation.

In Task 1b, we load the entire file into an `int[]` before starting the counting stopwatch. The timed region contains only the selected counting method, including its thread creation and final reduction. The raw integer payload alone needs

$$50\,000\,000 \times 4 = 200\,000\,000 \text{ bytes} \approx 190.73 \text{ MiB}. \tag{3.2.1}$$

Object headers, temporary loading allocations, threads, and runtime memory are additional costs. The supplementary Task 1b experiments include one-thread references so that a genuine speed-up relative to one thread can be calculated for each algorithm.

Peak memory is the process-wide maximum resident set size reported by macOS `/usr/bin/time -l`. On this system it is reported in bytes; we divide by 2^{20} to display MiB. It is resident physical memory, not reserved virtual address space and not just the managed heap. For the supplementary Task 1b measurements, the process peak includes array loading even though loading is excluded from the counting time. Consequently, those memory values describe the whole program's requirement, not an isolated allocation cost of the counting loop.

For Task 2, we measure the complete processing command on the actual USB volume. Before each of the three recorded configurations, we safely eject the drive, physically disconnect it, and reconnect it. Spotlight indexing is disabled and its status is checked after reconnection. We do not read or hash the part-file contents between reconnection and the timed run.

3.3 Speed-up and interpretation

For a fixed algorithm and input, we calculate

$$S(p) = \frac{T(1)}{T(p)}, \qquad E(p) = \frac{S(p)}{p}. \tag{3.3.1}$$

A speed-up above one means that the configuration is faster than its own one-thread reference. A value below one is a slowdown. For example, $S(p) = 3$ means three times faster, not a 300% reduction in elapsed time. For Task 1b, each version has its own reference; a ratio for the incorrect unsynchronised version is a timing observation only, not a valid speed-up of a correct computation.

We retain two real application snapshots, taken at approximately 15:43 and 16:42. The later snapshot is the primary source for Tasks 1a and 1c; the earlier snapshot provides a limited repeatability check. We do not describe two runs as a statistical confidence interval. The USB results previously shown by the menu have unverified storage/cache provenance and are excluded from the cold-cache USB conclusions. We preserve them in the raw archive rather than silently relabel them.

Background activity, runtime compilation, garbage collection, CPU scheduling, and the filesystem cache can influence timings. The internal-disk experiments did not force a cold cache. We therefore interpret

them as measurements of the complete implementation under the recorded conditions, not as isolated SSD throughput or proof of a purely CPU-bound workload.

4 Task 1a: partitioning one file among threads

4.1 Implementation and boundary ownership

We create p OS threads for $p \in \{1,2,4,8,16,32,64\}$. Each thread opens the same file independently and receives a nominal byte interval

$$a_i = \left\lfloor \frac{Li}{p} \right\rfloor, \qquad b_i = \left\lfloor \frac{L(i+1)}{p} \right\rfloor, \tag{4.1.1}$$

where L is the file size in bytes. No single reader loads the whole input first. Each worker uses its own 64 KiB byte buffer and parses integers directly, without allocating a string for every line.

A byte boundary can fall inside an integer. We assign a complete line to the worker whose nominal interval contains the first byte of that line. A worker that starts inside a line skips the remaining bytes of that line. The previous worker is allowed to finish a line that extends beyond its nominal end. This ownership rule prevents both skipping and double counting.

4.2 Source code: worker creation, boundaries, and counting

We include byte-identical snapshots of the submitted task sources, with their original line numbers. Keeping these snapshots inside the LaTeX project allows both a local TeX installation and the editor preview to compile the listings; the supplied synchronisation script verifies their hashes. The first listing creates each worker with its own captured index, computes the byte interval using 64-bit arithmetic, stores any exception in the corresponding error slot, and joins all workers before reading results.

```
        long fileLength = new FileInfo(filePath).Length;
        for (int i = 0; i < numberOfThreads; i++)
        {
            int workerIndex = i;
            long start = fileLength * workerIndex / numberOfThreads;
            long end = fileLength * (workerIndex + 1) / numberOfThreads;

            workers[i] = new Thread(() =>
            {
                try
                {
                    CountPart(
                        filePath,
                        start,
                        end,
                        fileLength,
                        out negativeCounts[workerIndex],
                        out positiveCounts[workerIndex],
                        out zeroCounts[workerIndex]);
                }
                catch (Exception exception)
                {
                    errors[workerIndex] = exception;
                }
            });
            workers[i].Start();
        }

        foreach (Thread worker in workers)
        {
            worker.Join();
        }
```

Listing 4.2.1 - Task 1a: creating workers for separate byte intervals and waiting for completion.

The complete `CountPart` method below opens an independent stream for a worker and performs the boundary adjustment. Both possible paths call the same range-counting routine, so a worker whose boundary falls inside a line cannot accidentally bypass the counting logic.

```
93      private static void CountPart(
94          string filePath,
95          long start,
96          long end,
97          long fileLength,
98          out long negative,
99          out long positive,
100         out long zero)
101     {
102         negative = 0;
103         positive = 0;
104         zero = 0;
105
106         using FileStream stream = new FileStream(filePath, FileMode.Open, FileAccess.Read, FileShare.Read);
107         long position = start;
108
109         if (start > 0)
110         {
111             stream.Position = start - 1;
112             int previousByte = stream.ReadByte();
113             stream.Position = start;
114
115             if (previousByte != '\n')
116             {
117                 using BufferedFileReader reader = new BufferedFileReader(stream);
118                 SkipToNextLine(reader, ref position, fileLength);
119                 CountRemainingLines(reader, ref position, end, fileLength, ref negative, ref positive, ref
    zero);
120                 return;
121             }
122         }
123
124         using BufferedFileReader bufferedReader = new BufferedFileReader(stream);
125         CountRemainingLines(bufferedReader, ref position, end, fileLength, ref negative, ref positive, ref
    zero);
126     }
```

Listing 4.2.2 - Task 1a: the complete per-worker file-opening and boundary-handling method.

The counting loop asks `ReadNumber` for a complete integer only while its current position is inside the nominal interval. The parser advances the shared position reference through the whole current line, including a possible crossing of the nominal end. Each output reference belongs to one worker's counter slot.

```
    private static void CountRemainingLines(
        BufferedFileReader reader,
        ref long position,
        long end,
        long fileLength,
        ref long negative,
        ref long positive,
        ref long zero)
    {
        while (position < end)
        {
            int value = ReadNumber(reader, ref position, fileLength);
            if (value < 0)
            {
                negative++;
            }
            else if (value > 0)
            {
                positive++;
            }
            else
            {
                zero++;
            }
        }
    }
```

Listing 4.2.3 - Task 1a: classifying complete numbers while their lines belong to the worker.

The outer counting loop starts another number only while `position < end`; the integer parser consumes the complete current line. A boundary immediately after a newline needs no adjustment. A boundary on a newline is skipped by the later worker, because the earlier worker owns the line ending there. We join every worker before reading its counters. Worker exceptions are recorded separately and cause the command to fail rather than return a partially successful result.

4.3 Measured results

Table 4.1 - Task 1a: primary session at 16:42.

Threads	Elapsed (ms)	Speed-up	Peak RSS (MiB)
1	3876	1.000	34.281
2	2262	1.714	34.156
4	1834	2.113	34.406
8	1505	2.575	34.406
16	1332	2.910	35.062
32	1353	2.865	36.469
64	1277	3.035	38.531

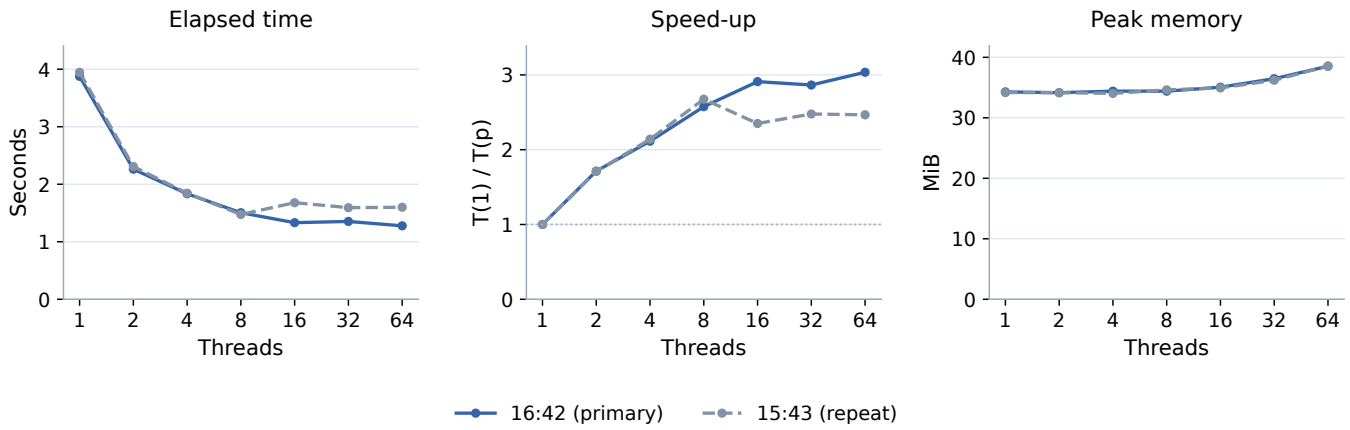


Figure 4.1 - Task 1a: elapsed time, speed-up relative to one thread, and peak resident memory. Solid curves are the 16:42 session; dashed curves are the 15:43 repeatability check.

The primary one-thread time is 3876 ms. It decreases to 1505 ms with eight threads, a speed-up of approximately 2.58. The lowest recorded primary time is 1277 ms at 64 threads, approximately 3.04 times faster than one thread. This remains far below ideal linear scaling. In the earlier session, eight threads were fastest at 1475 ms, while 64 threads took 1600 ms. We therefore observe a broad diminishing-returns region rather than a reproducible sharp optimum at one exact thread count.

4.4 Answers concerning scaling and memory

1. At which thread count does speed-up stop growing, and how does this relate to the CPU?

The major improvement occurs between one and eight threads, matching the machine's eight logical processors, of which four are performance cores. Beyond roughly 8–16 threads, improvements become much smaller and are inconsistent between sessions. In the primary session, 16, 32, and 64 threads take 1332, 1353, and 1277 ms respectively. The last value is the minimum observed, but the earlier session peaks at eight threads. We cannot honestly claim that the speed-up always stops exactly at eight. We can conclude that adding many more software threads than the eight hardware execution contexts does not provide proportional extra capacity.

2. Why do 32 and 64 threads not provide large gains? Are they slower?

They cannot create additional CPU cores. Extra runnable threads add scheduling and context-switching work, compete for caches and memory bandwidth, and can increase file-access and runtime overhead. Reading and parsing also share resources that do not scale with thread count. The serial portions and these overheads limit the benefit of more workers.

The measurements do not support the stronger statement that 32 and 64 threads are always slower. Both are slower than eight threads in the earlier session; 64 is faster than eight in the primary session. Smaller chunks, overlapping waits, and different scheduling/cache conditions may contribute, but identifying the

cause would require profiling. We report this variation instead of forcing the observations to match an expected curve.

3. Does peak memory depend on the number of threads? Why?

Yes, but the increase is modest here: the primary peak rises from approximately 34.28 MiB at one thread to 38.53 MiB at 64. Every worker adds a buffer, counter/error slots, runtime bookkeeping, and a stack with some resident pages. The explicit buffers alone occupy $p \times 64$ KiB, about 4 MiB at 64 workers. Reserved stack space is not the same as resident memory, so we do not multiply a nominal stack reservation by the thread count and equate it with RSS. This streaming implementation never needs to keep the full 352 MiB text file or 191 MiB integer payload resident in application buffers.

5 Task 1b: shared counters, locking, and local counters

5.1 Three implementations of the same counting operation

We load the dataset into an integer array before timing, then use eight threads because the runtime reports eight logical processors. The array is read-only during counting, so reading it concurrently does not require a lock. The important difference is where the mutable counters live.

5.2 Source code: all three counting methods

The following listings are the actual methods, including their returned counters. In the first version, the variables are declared outside the worker lambda and are shared captured state. Multiple workers therefore compete to update the same three values.

```
10      public static SignCounts CountUnsynchronized(int[] numbers, int workers)
11      {
12          long negative = 0, positive = 0, zero = 0;
13
14          RunWorkers(numbers.Length, workers, (start, end) =>
15          {
16              for (int i = start; i < end; i++)
17              {
18                  if (numbers[i] < 0) negative++;
19                  else if (numbers[i] > 0) positive++;
20                  else zero++;
21              }
22          });
23
24          return new SignCounts(negative, positive, zero);
25      }
```

Listing 5.2.1 - Task 1b, version 1: unsynchronised shared counters.

The locked version creates one lock object for the complete invocation. Every worker captures that same object; creating a different lock per worker would not protect shared counters. The classification and counter update are both inside the protected block.

```
    public static SignCounts CountWithLock(int[] numbers, int workers)
    {
        long negative = 0, positive = 0, zero = 0;
        object sync = new();

        RunWorkers(numbers.Length, workers, (start, end) =>
        {
            for (int i = start; i < end; i++)
            {
                lock (sync)
                {
                    if (numbers[i] < 0) negative++;
                    else if (numbers[i] > 0) positive++;
                    else zero++;
                }
            }
        });

        return new SignCounts(negative, positive, zero);
    }
```

Listing 5.2.2 - Task 1b, version 2: one shared lock around each classification and increment.

In the local-counter version, the three variables are declared inside the worker lambda. Only the owning worker changes them. After the helper has joined all threads, the calling thread reduces the result records into one final answer.

```
    public static SignCounts CountWithLocalCounters(int[] numbers, int workers)
    {
        SignCounts[] results = new SignCounts[workers];

        RunWorkers(numbers.Length, workers, (start, end, workerId) =>
        {
            long negative = 0, positive = 0, zero = 0;

            for (int i = start; i < end; i++)
            {
                if (numbers[i] < 0) negative++;
                else if (numbers[i] > 0) positive++;
                else zero++;
            }

            results[workerId] = new SignCounts(negative, positive, zero);
        });

        long totalNegative = 0, totalPositive = 0, totalZero = 0;

        foreach (var result in results)
        {
            totalNegative += result.Negative;
            totalPositive += result.Positive;
            totalZero += result.Zero;
        }

        return new SignCounts(totalNegative, totalPositive, totalZero);
    }
```

Listing 5.2.3 - Task 1b, version 3: worker-local counting followed by reduction.

All three methods share the same partitioning mechanism. Its two-argument callback overload delegates to the worker-ID overload shown below. The helper divides the array into half-open index intervals, captures each worker's index, starts the OS threads, and joins them. Consequently, the experimental difference is the counter-update strategy rather than a different partitioning or input-loading scheme.

```
80      private static void RunWorkers(
81          int length, int workers, Action<int, int> action)
82      {
83          RunWorkers(length, workers,
84              (start, end, id) => action(start, end));
85      }
86
87      private static void RunWorkers(
88          int length, int workers, Action<int, int, int> action)
89      {
90          Thread[] threads = new Thread[workers];
91
92          for (int id = 0; id < workers; id++)
93          {
94              int workerId = id;
95              int start = (int)((long)length * id / workers);
96              int end = (int)((long)length * (id + 1) / workers);
97
98              threads[id] = new Thread(() =>
99                  action(start, end, workerId));
100         }
101
102         foreach (var thread in threads)
103             thread.Start();
104
105         foreach (var thread in threads)
106             thread.Join();
107     }
```

Listing 5.2.4 - Task 1b: shared worker-launching helper and disjoint array index ranges.

The unsynchronised version updates shared captured variables. The locked version uses the same lock object for all workers and acquires it for every number. The local-counter version updates variables owned by one worker, writes one result record per worker, and adds those records after all workers have joined. The `lock` statement guarantees mutual exclusion for the protected block [4]; `Thread.Join` waits for worker termination before the final aggregation [5].

5.3 Measurements and correctness

Table 5.1 - Supplementary Task 1b: eight threads, with file loading excluded from counting time.

Version / run	Time (ms)	Negative	Positive	Zero	Total	Correct
Unsynchronized 1	306.201	6 446 577	6 483 152	30	12 929 759	No
Unsynchronized 2	261.485	7 042 223	7 039 905	30	14 082 158	No
Unsynchronized 3	268.229	6 086 430	6 303 483	30	12 389 943	No
Locked 1	1738.592	24 996 544	25 003 426	30	50 000 000	Yes
LocalCounters 1	131.284	24 996 544	25 003 426	30	50 000 000	Yes

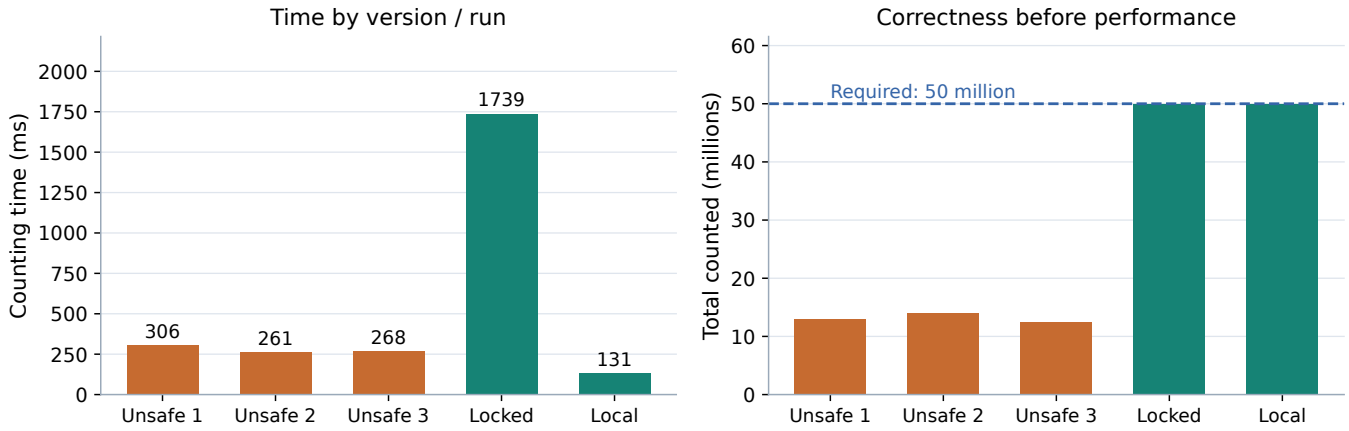


Figure 5.1 - Task 1b at eight threads: counting time and retained totals. All three unsynchronised repetitions are shown. A fast but incorrect count is marked as incorrect.

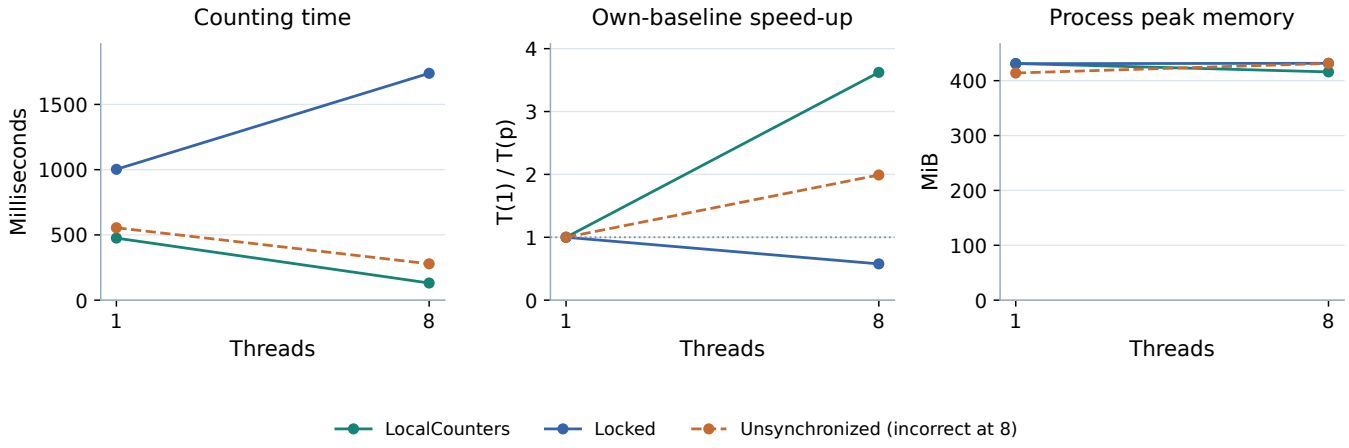


Figure 5.2 - Supplementary Task 1b measurements at one and eight threads. Each speed-up uses the same version's one-thread baseline. Memory is whole-process peak RSS, including array loading. The unsynchronised eight-thread timing is the mean of three incorrect runs.

4. Rank the versions by time. Which are correct? Explain each result.

In the supplementary eight-thread experiment, local counters are fastest at 131.284 ms. The three unsynchronised repetitions take 306.201, 261.485, and 268.229 ms. The locked version is slowest at 1738.592 ms. Thus the observed order is local counters, unsynchronised shared counters, then locked shared counters. The earlier application snapshot gives the same ordering, although its absolute times differ.

Local counters and locking both return the reference totals. The unsynchronised repetitions count only 12 929 759, 14 082 158, and 12 389 943 numbers. They lose approximately 72–75% of the updates. Their different totals on identical input demonstrate a race condition. The fact that all three retain the 30 zero values does not make the algorithm correct: zero updates are rare and happened not to collide in these runs.

Local counters avoid contention in the hot loop and perform only a small final reduction. Shared counters also cause cache-coherence traffic when cores compete to update the same memory locations, even if no lock is used. A lock makes the update safe but serialises this very small unit of work and repeatedly pays the cost of acquiring and releasing the lock. In this experiment the correct local-counter version is approximately 13.24 times faster than the locked version at eight threads.

The one-thread supplementary measurements make this distinction clearer: local counters take 475.585 ms at one thread and 131.284 ms at eight, giving approximately 3.62 times speed-up. Locking takes 1002.649 ms at one thread and 1738.592 ms at eight, so adding threads slows this version down. A one-thread unsynchronised execution is correct because there is no competing writer; that does not establish correctness once multiple writers are introduced.

5.4 How an update is lost

Table 5.2 - A possible interleaving of two increments of the same counter.

Step	Worker A	Worker B	Shared value
1	Reads 100		100
2		Reads 100	100
3	Computes 101		100
4		Computes 101	100
5	Writes 101		101
6		Writes 101	101

Two numbers were processed, but the counter increased only once. The increment is a read–modify–write operation; using a 64-bit counter does not make the whole operation atomic. A lock prevents this interleaving, while thread-local counters remove the competing writes. An atomic increment could also

preserve updates, but would still make all workers contend for shared counters and is outside the three required variants.

6 Task 1c: a producer and consumers

6.1 Pipeline, batching, and termination

We use one dedicated reader and p worker threads. The reader opens `numbers.txt`, reads lines, and groups them into batches of 10 000 strings. A bounded `BlockingCollection<List<string»` holds at most 100 batches. Workers repeatedly remove a batch, parse its integers, and update local counters.

6.2 Source code: queue lifecycle and both pipeline stages

The setup below creates the bounded queue and shared cancellation source. The producer signals completion in a `finally` block, so consumers are not left waiting indefinitely if reading fails. An error is also retained for reporting; cancellation releases blocked queue operations.

```
30          using BlockingCollection<List<string>> queue = new BlockingCollection<List<string>>(QueueCapacity);
31          using CancellationTokenSource cancellation = new CancellationTokenSource();
32
33          long[] negativeCounts = new long[workerCount];
34          long[] positiveCounts = new long[workerCount];
35          long[] zeroCounts = new long[workerCount];
36          Exception? producerError = null;
37          Exception?[] workerErrors = new Exception?[workerCount];
38
39          Thread producer = new Thread(() =>
40          {
41              try
42              {
43                  ProduceBatches(filePath, queue, cancellation.Token);
44              }
45              catch (OperationCanceledException) when (cancellation.IsCancellationRequested)
46              {
47              }
48              catch (Exception exception)
49              {
50                  producerError = exception;
51                  cancellation.Cancel();
52              }
53              finally
54              {
55                  queue.CompleteAdding();
56              }
57          });
```

Listing 6.2.1 - Task 1c: queue construction, producer error handling, and guaranteed completion signalling.

The complete producer method forms batches of `BatchSize = 10000` lines. The queue capacity is `QueueCapacity = 100` batches. Publishing a full batch transfers its list to consumers; the producer then allocates a new list, so it does not keep appending to a list already being processed elsewhere.

```
    private static void ProduceBatches(
        string filePath,
        BlockingCollection<List<string>> queue,
        CancellationToken cancellationToken)
    {
        using StreamReader reader = new StreamReader(filePath);
        List<string> batch = new List<string>(BatchSize);

        string? line;
        while ((line = reader.ReadLine()) is not null)
        {
            cancellationToken.ThrowIfCancellationRequested();
            batch.Add(line);

            if (batch.Count == BatchSize)
            {
                queue.Add(batch, cancellationToken);
                batch = new List<string>(BatchSize);
            }
        }

        if (batch.Count > 0)
        {
            queue.Add(batch, cancellationToken);
        }
    }
```

Listing 6.2.2 - Task 1c: reading lines, publishing full batches, and handling the final partial batch.

The complete consumer method removes batches through consuming enumeration, validates every integer, and updates its own output slots. It naturally finishes when no more batches can arrive and the queue has been drained. The enclosing worker catches errors and requests shared cancellation, following the same failure policy as the producer.

```
    private static void ConsumeBatches(
        BlockingCollection<List<string>> queue,
        CancellationToken cancellationToken,
        out long negative,
        out long positive,
        out long zero)
    {
        negative = 0;
        positive = 0;
        zero = 0;

        foreach (List<string> batch in queue.GetConsumingEnumerable(cancellationToken))
        {
            foreach (string line in batch)
            {
                if (!int.TryParse(line, out int value))
                {
                    throw new FormatException($"Invalid integer '{line}'.");
                }

                if (value < 0)
                {
                    negative++;
                }
                else if (value > 0)
                {
                    positive++;
                }
                else
                {
                    zero++;
                }
            }
        }
    }
```

Listing 6.2.3 - Task 1c: the complete consumer parsing and counting loop.

Once all participants have been constructed, we start the reader and consumers, join the reader, and then join every consumer. Waiting for the producer alone would be insufficient because queued batches could still be in progress.

```
        producer.Start();
        foreach (Thread worker in workers)
        {
            worker.Start();
        }

        producer.Join();
        foreach (Thread worker in workers)
        {
            worker.Join();
        }
```

Listing 6.2.4 - Task 1c: starting the pipeline and joining both producer and consumers.

This queue provides both thread-safe transfer and backpressure: an empty queue makes consumers wait, and a full queue makes the producer wait [3]. There is no busy loop checking for available work. Completion is different from cancellation: on normal completion, queued batches are still processed; on an error, the shared cancellation token releases blocked operations so that the program can report failure rather than deadlock. The producer also publishes the final partial batch if the input size is not a multiple of 10 000.

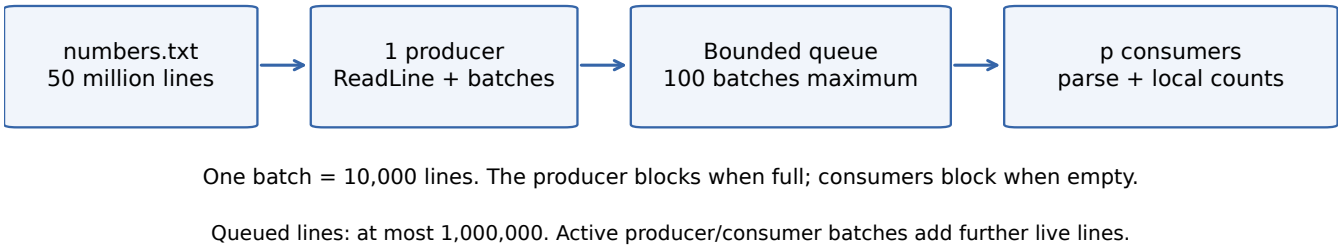


Figure 6.1 - Task 1c separates the sequential line reader from parallel integer parsing. The queue bounds waiting batches; additional batches can also be held by the producer and active workers.

6.3 Measured results and comparison

Table 6.1 - Task 1c: primary session at 16:42. The consumer count excludes the dedicated reader.

Consumers	Elapsed (ms)	Speed-up	Peak RSS (MiB)
1	1952	1.000	131.562
2	1853	1.053	53.359
4	1984	0.984	50.156
8	2248	0.868	59.062
16	2764	0.706	77.000
32	2902	0.673	98.594
64	3413	0.572	137.047

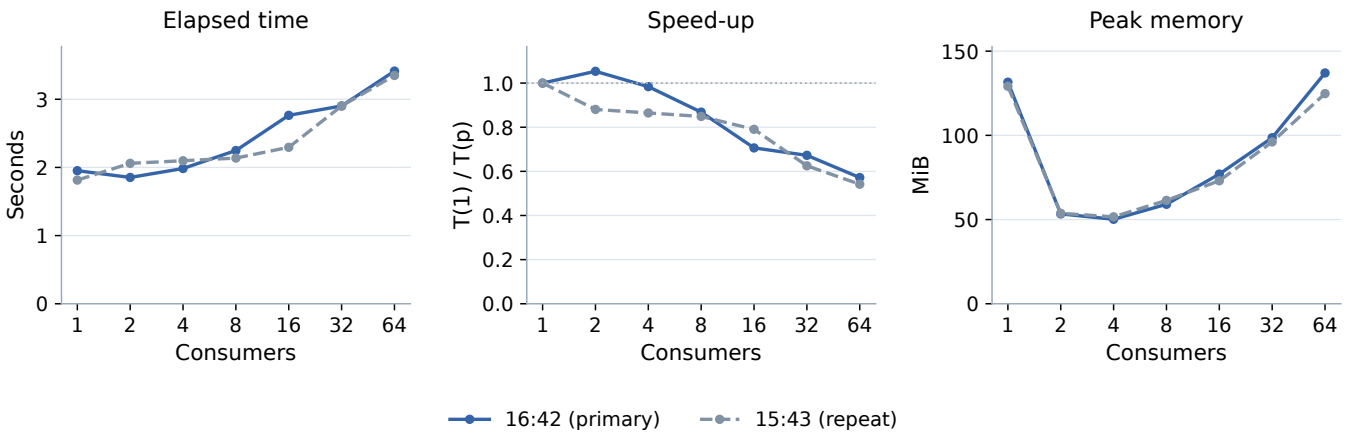


Figure 6.2 - Task 1c: time, speed-up relative to one worker, and peak resident memory in the two real saved sessions. The horizontal reference in the speed-up panel represents the one-worker time.

5. How does Task 1c compare with Task 1a, and when does it stop getting faster?

In the primary session, Task 1c takes 1952 ms with one worker, versus 3876 ms for Task 1a with one thread. Task 1c includes a separate producer, so "one worker" already means two active processing threads, in addition to the coordinating main thread. This is not a comparison of two single-threaded programs. Overlapping reading/line extraction with parsing can help, and the implementations also use different parsing and buffering paths.

Task 1c reaches its best primary time at two workers: 1853 ms, only about 1.05 times faster than its one-worker configuration. At four workers Task 1a is already faster: 1834 ms versus 1984 ms. With eight workers the gap grows to 1505 ms versus 2248 ms. Task 1c reaches 3413 ms at 64 workers, slower than its own one-worker configuration.

The sequential producer must read every line and allocate strings. Every batch also crosses a synchronised queue. Additional consumers cannot accelerate the producer and eventually add contention, scheduling work, and allocation/garbage-collection pressure. Task 1a instead lets workers read independent byte ranges and parse directly into local counters. In these measurements, Task 1c stops providing a useful improvement after about 1–2 workers; the earlier session is already fastest at one worker. The best Task 1c primary configuration is not the best configuration overall.

Peak memory is not monotonic: the primary session uses approximately 131.56 MiB at one worker, falls to 50.16 MiB at four, and rises to 137.05 MiB at 64. With few consumers the queue can accumulate strings; adding consumers can drain it faster. At large worker counts, active batches, thread resources, and runtime overhead grow. A bounded queue limits queued items, but does not force identical occupancy or identical RSS in every run.

6. What changes with one-line batches or an unbounded queue?

The current dataset produces exactly 5000 batches of 10 000 lines. One-line batches would require 50 million queue transfers, 10 000 times as many. That increases synchronisation, wake-up, bookkeeping, and small-allocation costs relative to the very cheap operation of classifying one integer. Smaller batches can improve balancing for expensive or irregular work, but a batch of one is a poor trade-off for this workload.

The present queue can hold at most one million waiting lines. This is not the total number of live lines: the producer may be building a batch, and each worker may hold another batch. With an unbounded queue, a fast producer can run far ahead and retain a large fraction of the 50 million strings, consuming far more memory than their compact integer representation. Garbage collection, memory pressure, and possibly swapping can then reduce throughput. Bounding the queue makes the producer wait and keeps the amount of queued work under control; it does not by itself remove all other sources of memory growth.

7 Task 2: eight files on the USB flash drive

7.1 Implementation and cold-cache procedure

We place the eight ordered part files on `/Volumes/ESD-USB`. A file is the unit of work. Each worker reads a whole assigned file with `File.ReadAllBytes`, parses the returned byte buffer, updates local counters, and proceeds to its next file. The library method returns a byte array containing the complete file [6]. One application-level read call does not imply that the operating system performs exactly one physical USB request.

7.2 Source code: file assignment, buffering, and parsing

The actual worker-creation loop assigns a half-open interval of file indices to each thread. The captured `workerIndex` chooses both the interval and the worker's output/error slots. All workers are joined before the caller checks errors and combines the counters.

```
        for (int i = 0; i < workerCount; i++)
        {
            int workerIndex = i;
            int firstFile = FileCount * workerIndex / workerCount;
            int lastFile = FileCount * (workerIndex + 1) / workerCount;

            workers[i] = new Thread(() =>
            {
                try
                {
                    CountFiles(
                        filePaths,
                        firstFile,
                        lastFile,
                        out negativeCounts[workerIndex],
                        out positiveCounts[workerIndex],
                        out zeroCounts[workerIndex]);
                }
                catch (Exception exception)
                {
                    errors[workerIndex] = exception;
                }
            });
            workers[i].Start();
        }

        foreach (Thread worker in workers)
        {
            worker.Join();
        }
```

Listing 7.2.1 - Task 2: balanced file assignment, worker startup, and completion.

The complete `CountFiles` method reads one assigned file at a time into one byte array. It does not construct a list of lines. Each buffer is parsed before the worker advances to its next file.

```
    private static void CountFiles(
        string[] filePaths,
        int firstFile,
        int lastFile,
        out long negative,
        out long positive,
        out long zero)
    {
        negative = 0;
        positive = 0;
        zero = 0;

        for (int fileIndex = firstFile; fileIndex < lastFile; fileIndex++)
        {
            byte[] bytes = File.ReadAllBytes(filePaths[fileIndex]);
            CountNumbers(bytes, ref negative, ref positive, ref zero);
        }
    }
```

Listing 7.2.2 - Task 2: whole-file byte-buffer reading for each assigned file.

The parser first skips separators (the preceding part of `CountNumbers`), then performs the sign, digit, and classification logic shown below. `checked` prevents silently wrapping an out-of-range value. The remainder of the method rejects non-separator characters after an integer, so malformed input is reported rather than counted as a successful partial read.

```
            int sign = 1;
            if (bytes[index] == (byte)'-' || bytes[index] == (byte)'+')
            {
                if (bytes[index] == (byte)'-')
                {
                    sign = -1;
                }

                index++;
            }

            if (index >= bytes.Length || bytes[index] < (byte)'0' || bytes[index] > (byte)'9')
            {
                throw new FormatException("An input file contains a non-integer value.");
            }

            long value = 0;
            while (index < bytes.Length && bytes[index] >= (byte)'0' && bytes[index] <= (byte)'9')
            {
                value = checked(value * 10 + bytes[index] - (byte)'0');
                index++;
            }

            int signedValue = checked((int)(sign * value));
            if (signedValue < 0)
            {
                negative++;
            }
            else if (signedValue > 0)
            {
                positive++;
            }
            else
            {
                zero++;
            }
```

Listing 7.2.3 - Task 2: actual sign parsing, checked digit accumulation, and classification inside CountNumbers.

With one thread, the worker processes eight files sequentially; with two, each handles four files; with four, each handles two. We use only the required 1, 2, and 4 threads for the cold-cache experiment. Using more than eight threads would leave some workers without an assigned file because there are only eight independent work units.

We disabled Spotlight indexing on the volume. Before each measurement, we safely ejected the device, unplugged it, and reconnected it. We checked that indexing remained disabled, then immediately ran the relevant configuration without previewing or reading the input files. All three commands use the

explicit absolute USB directory. We corrected the previous relative path `Volumes/ESD-USB`; without its leading slash it referred to a different location relative to the current working directory.

7.3 Measured results

Table 7.1 - Task 2: separate physical USB reconnect before every run; Spotlight disabled.

Threads	Elapsed (ms)	Speed-up	Peak RSS (MiB)	Total counted
1	4682.147	1.000	253.859	50 000 000
2	2847.351	1.644	302.703	50 000 000
4	2858.732	1.638	387.281	50 000 000

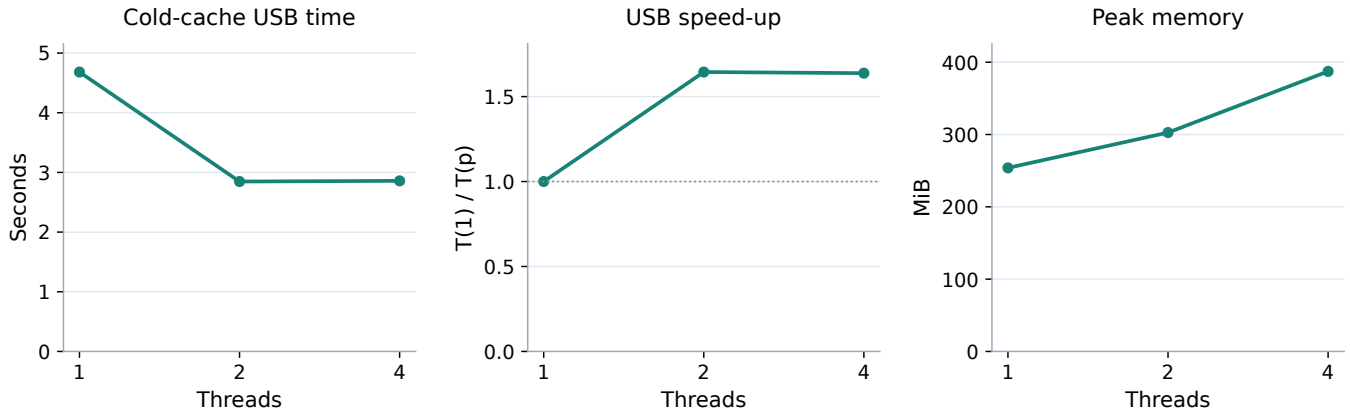


Figure 7.1 - Task 2: actual USB reads after a separate physical reconnect for every configuration. Speed-up uses the cold one-thread USB time. Peak memory includes complete-file byte buffers and runtime allocations.

Every cold-cache run returns the same three reference counters and a total of 50 million. The input volume and individual file contents are the same workload as in Task 1, but buffering and parsing implementations differ; this matters when comparing absolute times.

7.4 Answers about storage and concurrency

1. How does one USB thread compare with one internal-disk thread? Is the USB work CPU-bound or I/O-bound?

Task 1a on one thread takes 3876 ms in the primary internal-disk session. The cold USB run takes 4682.147 ms, a ratio of approximately 1.208. The USB command's application throughput is about 78.90 MB/s, calculated from the full input byte size and elapsed time. This is application throughput, including parsing and startup, not an isolated benchmark of the USB link.

The one-thread USB process uses 1.95 s of user CPU time and 0.12 s of system CPU time during 4.67 s of measured real time. Thus approximately 2.07 s of CPU time accounts for only about 44% of the elapsed time. This indicates a substantial non-CPU component, consistent with waiting for cold external-storage

reads. It does not mean that every remaining millisecond can be attributed exclusively to the flash hardware: scheduling and runtime behaviour also contribute.

We classify this cold USB workload as storage-sensitive, with a substantial I/O-wait component, rather than purely CPU-bound. We also avoid claiming that the whole difference from Task 1a is caused by the drive: Task 1a streams 64 KiB buffers, whereas Task 2 allocates whole-file buffers and uses a different parser. A stricter isolation of device cost would run the same Task 2 algorithm on both devices under matched cache conditions.

2. Do more USB threads help? Why is this different from a server waiting on the network?

The measured times are 4682.147, 2847.351, and 2858.732 ms for one, two, and four threads, giving speed-ups of 1.644 and 1.638 for two and four threads. Additional threads improve the measured throughput, but the improvement is sublinear: four workers do not deliver four times the one-worker performance. The four-thread run is only about 0.4% slower than the two-thread run. With one run per USB configuration, we treat this as a plateau rather than evidence of a statistically established regression.

All workers still share one flash device and its controller. Additional threads may overlap parsing with reading and may keep outstanding work available, but they do not multiply the drive's total bandwidth. Concurrent reads can also interfere with otherwise sequential access. The useful concurrency level therefore depends on the drive and workload, not simply on the fact that the operation involves I/O.

For a network server, different requests can be waiting for independent remote services or clients. While one request waits for a response, another may use the CPU or a different connection. Concurrency can hide these separate latency intervals before a shared resource becomes saturated. A single USB drive presents a much more direct shared-throughput limit. The lecture's general lesson remains valid, but it does not guarantee linear speed-up for concurrent reads from one physical device.

Peak RSS also increases when several complete input buffers are live together. Each file is about 44 MiB. Garbage collection may retain arrays from previously processed files until a collection occurs, so peak memory is not guaranteed to be exactly one file size per worker. The measured RSS includes those arrays, runtime memory, thread resources, and allocation history.

8 Answers to all control questions

1. What is the difference between a physical core and a logical processor? Where does it appear in our charts?

A physical core is a hardware execution core. A logical processor is an execution context exposed by the processor and OS to the scheduler. On a processor with simultaneous multithreading, several logical processors may share one physical core's execution resources. On our Apple M3, macOS exposes eight logical processors for eight physical cores: four performance and four efficiency cores. We therefore do not have an additional hardware doubling beyond eight.

In Task 1a, the substantial early gains and much smaller, variable gains beyond 8–16 software threads are consistent with this finite CPU capacity. In Task 1c, the reader and queue limit scaling long before all hardware contexts become useful. The charts show why core count is an upper-resource constraint rather than a guarantee that every algorithm reaches its optimum at exactly that count.

2. How do we ensure that no line is counted twice or skipped between two file parts?

We define ownership by the line's starting byte. A worker begins at its nominal boundary only if that boundary follows a newline. Otherwise it advances to the next complete line. At the other end, it finishes any line whose first byte it already owns, even when that line crosses the nominal boundary. The next worker skips that same partial line. These two rules fit together: the earlier worker owns the crossing line, and the later worker owns the next line. We also check all three sign counts against the common reference and verify that the ordered part files reproduce the main file.

3. How do unsynchronised shared counters lose updates, step by step?

First, one worker reads a counter value. Before it writes back, another worker reads the same old value. Each computes the old value plus one. Both then write the same new value, so one increment overwrites the effect of the other. Table 5.2 shows a concrete interleaving from 100 to 101 instead of 102. This remains possible even when individual reads and writes of the counter fit in a machine word, because the whole increment contains multiple operations. The differing incorrect totals in our three repetitions demonstrate the practical consequence.

4. Why are per-thread counters faster than shared counters protected by a lock?

Each worker can increment its own variables without acquiring a shared lock on every number. The hot loops run independently, and the final reduction costs only a few additions per worker after joining. A shared lock serialises the protected block and can force waiting, ownership transfers, and scheduling work. Even an extremely short critical section becomes expensive when we enter it 50 million times. Our eight-thread local-counter measurement is 131.284 ms, versus 1738.592 ms for locking, while both produce the same correct totals.

5. What does I/O-bound mean for a USB drive and for a network server? Why can their response to concurrency differ?

I/O-bound means that waiting for input/output, rather than available CPU computation alone, limits progress. For our cold USB run, the process spends much less CPU time than real elapsed time and all reads share the same device. Several threads cannot create additional flash bandwidth. For a server, waiting can concern many independent requests, remote computers, or network round trips, so concurrency can use otherwise idle CPU time while those waits overlap.

The statement that concurrency helps only the network case is a useful simplification, not a universal rule. USB concurrency can also help when it overlaps parsing with reading, hides device latency, or improves the controller's utilisation; our actual speed-up must be read from the measured curve. It eventually encounters a shared device limit. A network service can likewise stop benefiting once its CPU, connection limits, or network bandwidth are saturated.

6. How does static partitioning differ from taking work from a queue? When is each preferable?

Static partitioning assigns each worker a fixed range before processing starts. Tasks 1a and 2 use this approach. It has little coordination overhead and is effective when work units are sufficiently uniform. Its weakness is imbalance: a worker that finishes early cannot automatically take a portion of a slower worker's assignment. Equal bytes also do not necessarily mean exactly equal parsing time.

Queue-based scheduling assigns the next available unit to whichever worker finishes and requests more work. Task 1c can therefore balance consumers dynamically when batch costs differ. It also adds a producer, queue operations, synchronisation, and a need for termination/backpressure rules. We prefer static partitioning for cheap, regular work where independence is easy to arrange, and a bounded queue for

streaming pipelines or work with variable processing costs. Our measurements show that dynamic scheduling is not automatically faster when the producer itself is the bottleneck.

9 Build, run, and inspect the implementation

9.1 Source organisation and command-line entry point

The submitted C# sources are in `code/`. `TaskOneA.cs`, `TaskOneB.cs`, `TaskOneC.cs`, and `TaskTwo.cs` implement the algorithms; `Program.cs` provides the interactive menu and metric collectors; `Drawer.cs` builds the browser dashboard. We provide `documentation/reproduce/lab1.sh` as one command-line entry point for selecting tasks and configurations without navigating the menu. The implementation remains organised as separate file-based C# modules, launched by the entry point.

From the laboratory directory, we build before measuring:

```
cd Lab1.Concurrency-in-practice
bash documentation/reproduce/lab1.sh build
```

Listing 9.1.1 - Preparing the file-based executables.

The following examples select every task explicitly. The second argument is the number of processing threads or consumers. For Task 1b, the version and repetition label are additional arguments; loading the integer array remains outside the internal stopwatch.

```
bash documentation/reproduce/lab1.sh task1a 8
bash documentation/reproduce/lab1.sh task1b 8 Unsynchronized 1
bash documentation/reproduce/lab1.sh task1b 8 Unsynchronized 2
bash documentation/reproduce/lab1.sh task1b 8 Unsynchronized 3
bash documentation/reproduce/lab1.sh task1b 8 Locked 1
bash documentation/reproduce/lab1.sh task1b 8 LocalCounters 1
bash documentation/reproduce/lab1.sh task1c 2
bash documentation/reproduce/lab1.sh task2 1 /Volumes/ESD-USB
```

Listing 9.1.2 - Examples for all four tasks.

For Tasks 1a and 1c, we repeat the command with 1, 2, 4, 8, 16, 32, and 64 workers. For Task 1b, eight workers satisfy the required logical-processor setting; one worker is an additional baseline used only to calculate scaling. For Task 2, we run 1, 2, and 4 separately, with a physical reconnect before each, rather than putting all configurations in an uninterrupted loop.

If the existing input is absent, menu option g generates the main file and the eight parts. Re-generating input changes the experiment, so we keep the original files for comparison with the recorded hashes. The

USB must contain the eight part files before beginning cold-cache measurements; copying and checking them belongs before the eject/reconnect sequence.

9.2 Measuring a processing command on macOS

```
cd code
/usr/bin/time -l -o task1a-8.time.txt \
  dotnet artifacts/TaskOneA/TaskOneA.dll 8
```

Listing 9.2.1 - An example of whole-process timing and peak-memory capture.

The `real` value is wall-clock time; `user` and `sys` record CPU time. We retain the maximum resident set size and the printed sign totals. For Task 1b, we use the benchmark's internal `ELAPSED_MS` value for counting time and the external process peak for memory. Timing a complete `dotnet run` compilation command would add build overhead and would not reproduce that counting metric.

9.3 USB preparation and access

```
diskutil list external physical
ls /Volumes
diskutil info /Volumes/ESD-USB
mdutil -i off /Volumes/ESD-USB
mdutil -s /Volumes/ESD-USB
cd ~
diskutil eject /Volumes/ESD-USB
# Physically unplug and reconnect, then measure one configuration.
```

Listing 9.3.1 - Checking and preparing the USB volume.

The absolute path begins with `/Volumes`. A terminal whose working directory is inside the USB volume can prevent safe ejection; changing to the home directory releases that reference. We do not force-unmount the drive. If the operating system requires administrative privileges for indexing control, that preparation is performed with the appropriate local authorisation before the timed experiment.

9.4 Dashboard and evidence

Launching `dotnet run -file Program.cs` from `code/` opens the original menu. After collecting metrics, option `d` passes all four metric sets to `Drawer.cs`, which generates an offline HTML report and opens it in a browser. Task 1b has individual run checkboxes. Unchecking `Locked` removes it from both graphs and recalculates their scales, making the smaller timing differences visible. The exact-value table retains every original measurement.

The report uses vector plots and searchable source listings for readable results and code. We retain the original real dashboard snapshots, CSV/JSON measurements, dataset manifest, and per-run timing/output logs in `report/data/`. The accompanying scripts reproduce the figures and processing commands.

10 Conclusions

We found that the organisation of the work matters at least as much as the number of threads. Task 1a gains substantially from partitioning a single file across workers: the primary time decreases from 3876 to 1277 ms, but returns beyond 8–16 threads are small and unstable between sessions. We interpret this as CPU-sensitive parsing combined with shared memory, I/O, scheduling, and runtime costs; without matched cold-cache tests or profiling, we do not describe the internal-disk task as purely CPU-bound.

We classify Task 1b as an in-memory CPU/memory-bound counting workload: input loading is outside the timed region, and the counting loops perform no file I/O. It is our clearest CPU-side comparison, although memory bandwidth and shared-cache traffic can still limit scaling. Per-thread counters are both correct and fastest in our eight-thread measurements. The locked version is correct but much slower because it serialises millions of small updates. The unsynchronised version returns different incorrect totals on the same input. Its elapsed time therefore cannot be used as evidence of a successful correct optimisation.

Task 1c demonstrates a different scaling limit. Its bounded queue prevents unlimited producer lead, but the single reader, string allocation, and queue transfers restrict throughput. The primary optimum occurs at two consumers, and 64 consumers are slower than one. Memory depends on queue occupancy and active batches as well as thread count, which explains why its peak RSS is not monotonic.

The cold USB experiment introduces a substantial storage-wait component: one thread needs 4682.147 ms for the same dataset, compared with 3876 ms in the primary one-thread Task 1a session. The USB run uses only about 2.07 s of CPU time during 4.67 s of real time. Additional workers achieve the measured speed-ups of 1.644 and 1.638, while continuing to share one flash device. We therefore distinguish latency overlap from multiplying storage bandwidth, and we keep the USB conclusions separate from earlier measurements with unverified cache provenance.

Our practical recommendation for this workload is to use independent counters and a small, measured degree of concurrency, to verify all counters before interpreting timings, and to keep cache conditions explicit. The best worker count is an experimental result, not a value that can be inferred from core count alone.

Bibliography

[1] Network Programming. *Laboratory Work 1: Concurrency in Practice*. Course handout supplied with this laboratory, four pages. Requirements, task questions, control questions, and grading criteria.

[2] Apple. *MacBook Air (15-inch, M3, 2024): Technical Specifications*. https://support.apple.com/en-us/118552. Accessed 25 October 2026.

[3] Microsoft. *BlockingCollection Overview*. .NET documentation. https://learn.microsoft.com/en-us/dotnet/standard/collections/thread-safe/blockingcollection-overview. Accessed 25 October 2026.

[4] Microsoft. *C# Language Specification: Statements, Section 13.13, the lock statement*. https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/language-specification/statements. Accessed 25 October 2026.

[5] Microsoft. *Thread.Join Method*. .NET API documentation. https://learn.microsoft.com/en-us/dotnet/api/system.threading.thread.join. Accessed 25 October 2026.

[6] Microsoft. *File.ReadAllBytes Method*. .NET API documentation. https://learn.microsoft.com/en-us/dotnet/api/system.io.file.readallbytes. Accessed 25 October 2026.